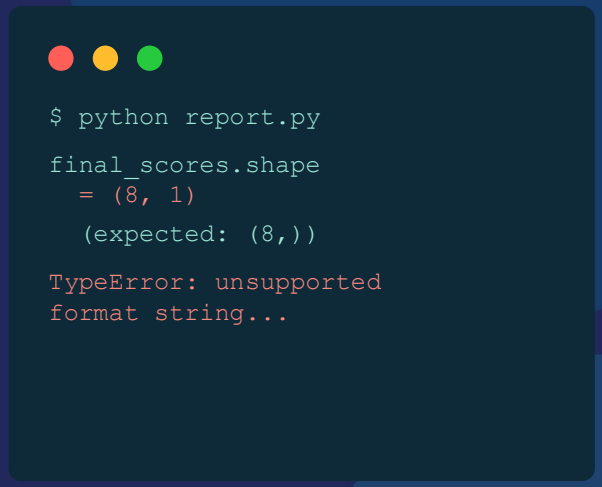
Debugging Exercise

Shapes, Loops & Breakpoints

Computing student final scores with NumPy — and finding the first failing student



The Setup

Two files, two bugs, one grade report

`student_scores.py`

Two functions: compute_final_scores() (NumPy matrix math) and find_first_failing_student() (a for-loop with a break). Each has one bug.

`report.py`

The runnable script (don't edit this!) that builds an 8-student roster, computes scores, and reports the first student below 70.0.

The Data

- scores: shape (8, 4) — 8 students x HW1/HW2/Midterm/Final
- weights: shape (4,) — 0.15 / 0.15 / 0.30 / 0.40 (sums to 1.0)
- PASSING_GRADE = 70.0

Your Mission

Fix 2 bugs in student_scores.py: a NumPy shape problem in compute_final_scores(), and a for-loop break condition in find_first_failing_student(). Make python report.py run cleanly and report the correct student.

The Roster

8 students, 4 assignments — who's at risk?

Student	HW1	HW2	Midterm	Final
Alice	85	90	88	92
Bob	78	82	75	80
Carla	92	95	91	96
Dinesh	60	65	55	62
Elena	88	84	90	87
Farid	70	68	72	71
Grace	95	98	94	97
Hugo	55	60	58	63

Assignment Weights

```
HW1: 15%      HW2: 15%
Midterm: 30%   Final: 40%
weights = [0.15, 0.15, 0.30, 0.40]
```

At-Risk Students

Dinesh and Hugo both have low scores across the board. The report should find Dinesh FIRST, since he comes earlier in roster order (index 3 vs. index 7).

Bug 1: compute_final_scores()

A NumPy shape problem hiding behind a reshape()

```
def compute_final_scores(scores, weights):
    n_students, n_assignments = scores.shape

    weights_col = weights.reshape(
        n_assignments, 1)
    final_scores = scores @ weights_col

    return final_scores
```

(8,4) @ (4,1) -> (8,1)

We wanted shape (8,) — one number per student — but the column-vector reshape adds an extra dimension, giving (8, 1) instead.

What's Going Wrong

weights.reshape(4, 1) turns a (4,) vector into a (4,1) column. scores @ weights_col then yields (8,4) @ (4,1) -> (8,1), not (8,).

Ask Yourself

What shape does weights already have? Do we even need the reshape() to get scores @ weights -> (8,)?

Fix

-> (8,)

Investigating with .shape

Shapes don't lie — even when code doesn't crash

Key Habit: whenever a NumPy result looks wrong (or the script crashes downstream), print .shape at every step — don't wait for an error.

```
import numpy as np
from student_scores import compute_final_scores
import report

print(report.scores.shape)     # (8, 4)
print(report.weights.shape)    # (4,)

weights_col = report.weights.reshape(4, 1)
print(weights_col.shape)       # (4, 1)  <- already suspicious!

result = report.scores @ weights_col
print(result.shape)            # (8, 1)  <- not (8,) !
```

Bug 2: find_first_failing_student()

A for-loop break condition that's always true

```
for i, name in enumerate(student_names):
    score = final_scores[i]

    # >>> BREAKPOINT HERE <<<

    if i >= 0:
        first_failing_index = i
        first_failing_name = name
        break

return first_failing_index,
first_failing_name
```

What's Going Wrong

i >= 0 is TRUE on the very first iteration (i=0) and every iteration after. The loop breaks immediately at Alice — who is passing with grade 89.45!

Ask Yourself

We want to break ONLY when this student is failing. What comparison between grade and PASSING_GRADE expresses 'this student failed'?

Fix

Investigating with a Breakpoint

Step through the loop, one student at a time

`i = 0`

Alice

`score = 89.45`

PASS

with fixed condition: continue loop

→

`i = 1`

Bob

`score = 78.50`

PASS

with fixed condition: continue loop

→

`i = 2`

Carla

`score = 93.75`

PASS

with fixed condition: continue loop

→

`i = 3`

Dinesh

`score = 60.05`

FAIL

BREAK HERE ✓

Debugger Watch Panel (at breakpoint, each iteration)

```
i                -> 0, 1, 2, 3, ...
name             -> 'Alice', 'Bob', 'Carla', 'Dinesh', ...
score            -> 89.45, 78.50, 93.75, 60.05, ...
score < PASSING_SCORE  -> False, False, False, True  <- stop here
```

Expected Output

What success looks like after both fixes

```
=== Final Scores ===
final_scores.shape = (8,)  (expected: (8,))
Alice: 89.45
Bob: 78.50
Carla: 93.75
Dinesh: 60.05
Elena: 87.60
Farid: 70.70
Grace: 95.95
Hugo: 59.85

=== First Failing Student (score < 70.0) ===
First failing student: Dinesh (index 3), score =
60.05
```

Sanity Checks

Shape
final_scores.shape == (8,) — a plain 1D array, not (8, 1).

No crash
Each grade prints as a normal float with .2f formatting.

First failure
Dinesh (index 3) is reported, not Hugo (index 7).

Edge case
If everyone passes, output is 'No failing students found.'

Reflection Questions

1. scores @ weights and scores @ weights.reshape(-1, 1) both run without error but give different shapes. Why is checking .shape important even when code doesn't crash?

2. if i >= 0: is always True for enumerate() starting at 0 — yet Python never raised an error for this 'useless' condition. What does that tell you about relying on crashes alone?

3. When would you reach for a debugger breakpoint instead of print() to investigate a loop like this one?

Remember: don't edit report.py — only fix student_scores.py